Unió-Holvan adatszerkezet

Szerző: Nikházy László
laszlo.nikhazy@gmail.com

1. Partíció fogalma

Legyen adott egy véges alaphalmaz

$$U = \{1, 2, \ldots, n\}.$$

1.1. definíció. *Az* U *egy* ***partíciója*** *olyan felbontás*

$$U = S_1 \cup S_2 \cup \cdots \cup S_k,$$

amelyre teljesül:

- $S_i \neq \emptyset$ *minden* i*-re,*
- $S_i \cap S_j = \emptyset$*, ha* $i \neq j$*,*
- *minden* $u \in U$ *pontosan egy* S_i *halmazban szerepel.*

Ez intuitíven azt jelenti, hogy az elemeket *diszjunkt halmazokba* csoportosítjuk. Minden elemhez hozzárendelhetünk egy címkét: melyik halmazhoz tartozik. A partíció halmazait sokszor *osztályoknak* vagy *ekvivalenciaosztályoknak* is nevezzük.

Példa:

$$U = \{1, 2, 3, 4, 5, 6\},$$
$$\big\{\{1, 4\}, \{2, 3, 6\}, \{5\}\big\}$$

2. Unió–Holvan – absztrakt adatszerkezet

Az **Unió–Holvan** (Disjoint Set Union, DSU) adatszerkezet egy adott alaphalmaz partícióját tárolja, és lehetővé teszi a halmazok hatékony lekérdezését és összevonását.

Kiindulásként tekintsünk egy

$$U = \{1, 2, \ldots, n\}$$

alaphalmazt. Az adatszerkezet minden időpillanatban ennek a halmaznak egy partícióját reprezentálja, azaz az elemek diszjunkt halmazokra vannak felosztva.

Támogatott műveletek

- **`holvan(x)`**: Meghatározza, hogy az `x` elem melyik halmazhoz tartozik. A gyakorlatban nem magát a teljes halmazt adjuk vissza, hanem annak egy reprezentánsát (például egy azonosítót vagy egy kijelölt gyökérelemet).
- **`unio(x,y)`**: Egyesíti azt a két halmazt, amelyben `x` és `y` található. Ha a két elem már eleve ugyanabban a halmazban van, akkor a művelet nem változtat a partíción.

Ennek megfelelően az adatszerkezet dinamikusan karbantartja az elemek felosztását, miközben lehetővé teszi annak gyors eldöntését, hogy két elem azonos halmazban van-e.

3. Unió-Holvan – megvalósítások

3.1. Naiv tömbös reprezentáció

Az Unió–Holvan adatszerkezet legegyszerűbb megvalósítása egy tömb segítségével történik. Az ötlet az, hogy minden elemhez közvetlenül eltároljuk, melyik halmazhoz tartozik.

Reprezentáció:

Legyen adott egy `U = {1, 2, ..., n}` alaphalmaz. Tárolunk egy `id` tömböt, ahol

`id[x] = x` halmazazonosítója.

A halmazazonosító lehet például a halmaz egy kijelölt eleme. Az azonos `id` értékű elemek ugyanahhoz a halmazhoz tartoznak.

Példa:

Tegyük fel, hogy az alaphalmaz: `U = {1, 2, 3, 4, 5, 6}`. A partíció legyen: `{{1, 2}, {3, 4}, {5, 6}}`.

Ekkor a `id` tömb például a következő lehet:

x	1	2	3	4	5	6
id[x]	1	1	3	3	5	5

Az 1-es és 2-es elem ugyanahhoz a halmazhoz tartozik, mivel `id[1] = id[2] = 1`, a 3-as és 4-es elem szintén azonos halmazban van, míg az 5-ös és 6-os egy harmadik halmazt alkot.

Műveletek:

`holvan(x)` esetén egyszerűen visszaadjuk a `id[x]` értékét.

Ez közvetlen tömbhozzáférés, tehát a művelet időigénye `O(1)`.

`unio(x,y)` végrehajtásakor először meghatározzuk

`a = id[x],  b = id[y].`

Ha `a = b`, akkor `x` és `y` már ugyanabban a halmazban vannak, így nincs teendő.

Ha `a` $\neq$ `b`, akkor az összes olyan `i` elemre, amelyre `id[i] = b`, beállítjuk, hogy `id[i] = a`.

Ez a teljes tömb átvizsgálását igényli.

Komplexitás:

- `holvan(x):O(1)`
- `unio(x,y):O(n)`

Hátrány:

Ha `O(n)` `unio` műveletet hajtunk végre, akkor a teljes futásidő $O(n^2)$ lehet. Ez nagy elemszám esetén már nem elfogadható, ezért hatékonyabb reprezentációra van szükség.

3.2. Tömb + listák reprezentáció

A naiv tömbös megoldás továbbfejleszthető úgy, hogy nemcsak az `id` tömböt tároljuk, hanem minden halmazhoz külön listát is vezetünk az elemeiről.

Reprezentáció:

- Minden elemhez: `id[x]` = halmazazonosító.
- Minden halmazhoz egy lista az elemeiről.

Formálisan:

$$\texttt{lista[h]} = \{\texttt{x} \mid \texttt{id[x]} = \texttt{h}\}.$$

Példa:

Legyenek az elemek: `1, 2, 3, 4, 5, 6`.

Tegyük fel, hogy a halmazok:

$$\texttt{lista[1]} = \{1, 2\}, \quad \texttt{lista[3]} = \{3, 4\}, \quad \texttt{lista[5]} = \{5, 6\}.$$

Az `id` tömb ekkor:

x	1	2	3	4	5	6
id[x]	1	1	3	3	5	5

Műveletek:

`holvan(x)` esetén egyszerűen visszaadjuk az `id[x]` értékét, tehát az időigény továbbra is `O(1)`.

`unio(x,y)` esetén legyen `a = id[x]` és `b = id[y]`.

Ha `a = b`, nincs teendő.

Ha `a` $\neq$ `b`, akkor minden `v` $\in$ `lista[b]` elemre:

- beállítjuk, hogy `id[v] = a`,
- hozzáadjuk `v`-t a `lista[a]` listához.

Végül a `lista[b]` törölhető.

Komplexitás:

- `holvan(x):O(1)`
- `unio(x,y):O(|`halmaz$_b$`|)`

Az unió költsége tehát arányos a beolvasztott halmaz méretével.

3.2.1. Small-to-large merging és amortizált futásidő

A módszer tovább gyorsítható az úgynevezett *small-to-large merging* technikával.

Small-to-large merging (kisebb a nagyobba):

Unió során mindig a kisebb halmaz elemeit másoljuk át a nagyobba, és a nagyobb halmaz marad a reprezentáns.

Miért hatékony?

Ha egy elem átkerül egy másik halmazba, akkor az új halmaz mérete legalább kétszer akkora, mint az előző volt.

Egy halmaz mérete legfeljebb n, ezért egy elem legfeljebb $\log_2 n$ alkalommal válthat halmazt.

Összesített futásidő:

Ha legfeljebb n unió műveletet hajtunk végre, akkor az összes másolás száma legfeljebb $O(n \log n)$.

Így az unió művelet amortizált költsége $O(\log n)$.

Amortizált költség fogalma

Nem egyetlen művelet legrosszabb esetét vizsgáljuk, hanem sok művelet együttes, átlagos költségét.

Bár egy konkrét unió drága lehet, egy elem összesen csak kevésszer mozog, ezért a hosszú műveletsor átlagos költsége alacsony marad.

3.3. Fa-alapú reprezentáció

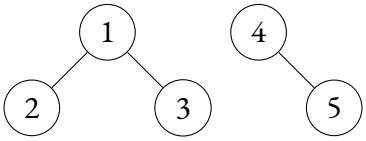
A leghatékonyabb klasszikus megvalósításban az Unió–Holvan adatszerkezetet *erdőként* (diszjunkt fák halmazaként) reprezentáljuk.

Reprezentáció:

- minden elemhez eltároljuk a szülőjét: `szulo[x]`,
- ha `szulo[x] = x`, akkor `x` gyökér,
- egy halmaz azonosítója a fa gyökere.

A `holvan(x)` művelet a gyökér megkeresése, az `unio(x,y)` pedig két fa összekötése.

Szemléltetés



Halmazok:

- Gyökér 1: `{1, 2, 3}`
- Gyökér 4: `{4, 5}`

x	1	2	3	4	5
szulo[x]	1	1	1	4	4

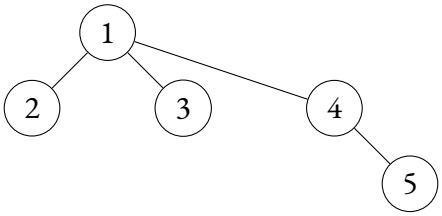
`unio(3,5)` példa

Unió előtt:

`holvan(3) = 1,  holvan(5) = 4`

Az egyik gyökeret a másik alá kötjük (például 4-et 1 alá).

Unió után:



x	1	2	3	4	5
szulo[x]	1	1	1	1	4

Például `holvan(5)` útja: $5 \to 4 \to 1$.

3.3.1. Műveletek és komplexitás

A fa-reprezentációban minden halmaz egy gyökeres fa, és a halmaz azonosítója a gyökér.

`holvan(x)` működése:

A `holvan(x)` művelet során az `x` csúcsból indulva felfelé haladunk a `szulo` mutatókon keresztül, amíg olyan csúcsot nem találunk, amelyre `szulo[x] = x`. Ez a csúcs a gyökér, tehát a halmaz reprezentánsa.

Ha a fa mélysége `h`, akkor legrosszabb esetben `h` lépést kell megtennünk, ezért a művelet költsége `O(h)`, azaz arányos a fa mélységével.

`unio(x,y)` működése:

Az unió művelet először meghívja a `holvan(x)` és `holvan(y)` függvényeket, így meghatározza a két elem gyökerét.

Ha a két gyökér azonos, akkor nincs teendő. Ha különböznek, akkor az egyik gyökeret a másik alá kötjük, azaz egy új élt adunk a két fa közé.

Mivel az unió két gyökérkeresést és egy konstans idejű összekapcsolást tartalmaz, a költsége szintén `O(h)`, ahol `h` a fák mélysége.

A probléma:

Megfelelő stratégia nélkül a fák nagyon kiegyensúlyozatlanná válhatnak. Ha például mindig egy egycsúcsos fához kötjük egy már meglévő fa gyökerét, akkor a szerkezet könnyen lánccá alakulhat:

$$1 \leftarrow 2 \leftarrow 3 \leftarrow 4 \leftarrow \cdots \leftarrow n$$

Ebben az esetben a fa mélysége $n-1$, tehát a `holvan` és az `unio` műveletek legrosszabb esetben `O(n)` időigényűek.

Ez indokolja, hogy az unió műveletet valamilyen kiegyensúlyozó stratégiával (rang vagy méret szerint) egészítsük ki.

3.3.2. Rang szerinti unió

A rang szerinti unió egy olyan stratégia, amely segít megtartani a fa kiegyensúlyozottságát, így a műveletek hatékonyak maradnak.

Ötlet:

Minden gyökérhez tárolunk egy **rang** értéket, ami a fa becsült mélységét jelzi. Amikor két halmazt uniózunk, a rangot használjuk arra, hogy a kisebb mélységű fát a nagyobb alá kössük.

Ezáltal a fa mélysége nem nő feleslegesen, és a láncszerű szerkezet kialakulásának esélye csökken.

Rang frissítése:

- Kezdetben minden fa rangja 1.

- Ha két azonos rangú fát egyesítünk, az egyik gyökér lesz az új gyökér, és ennek a rangja eggyel nő.
- Ha különböző rangú fákat kötünk össze, a rang nem változik.

Eredmény:

Az így kialakított fák mélysége legfeljebb `O(log n)` lesz, így a `holvan` és `unio` műveletek hatékonyak maradnak.

3.3.3. Méret szerinti unió

Egy másik hasonló stratégia a **méret szerinti unió**, amely a halmazok elemszámát használja a kiegyensúlyozásra.

Ötlet:

Minden gyökérhez tároljuk a halmaz méretét. Amikor két halmazt egyesítünk, a kisebb elemszámú halmaz gyökerét a nagyobb alá kötjük.

Mélység elemzés:

Ha egy elem mélysége nő, az azt jelenti, hogy a halmaz, amelyhez tartozik, legalább kétszeresére nőtt. Mivel a halmaz mérete legfeljebb `n`, egy elem legfeljebb $\log_2 n$ alkalommal kerülhet mélyebb szintre.

Komplexitás:

Ennek eredményeként a `holvan` és `unio` műveletek amortizált időigénye `O(log n)` műveletenként.

Kód (méret szerinti unió):

```
vector<int> szulo, meret;

void init(int n) {
    meret.assign(n + 1, 1);
    szulo.resize(n + 1);
    iota(szulo.begin(), szulo.end(), 0);
}

int holvan(int x) {
    while (szulo[x] != x)
        x = szulo[x];
    return x;
}

void unio(int a, int b) {
    a = holvan(a);
    b = holvan(b);
    if (a == b) return;
    if (meret[a] < meret[b]) swap(a, b);
    szulo[b] = a;
    meret[a] += meret[b];
}
```

3.3.4. Útösszenyomás (path compression)

Az útösszenyomás technikával a `holvan(x)` művelet amortizált költségét tudjuk szignifikánsan csökkenteni.

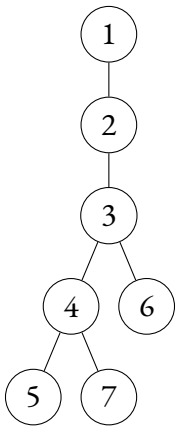
Alapötlet:

Amikor meghívjuk a `holvan(x)` függvényt, és felfelé haladunk a gyökér felé, akkor nemcsak megkeressük a gyökeret, hanem a bejárt csúcsokat közvetlenül a gyökérhez kötjük.

Másképp fogalmazva: a keresés során érintett csúcsok szülője a gyökér lesz.

Ez azt jelenti, hogy a fa szerkezete minden `holvan` hívás után „javul”: a hosszú utak rövidebbé válnak, a fa fokozatosan ellaposodik.

Útösszenyomás előtt:



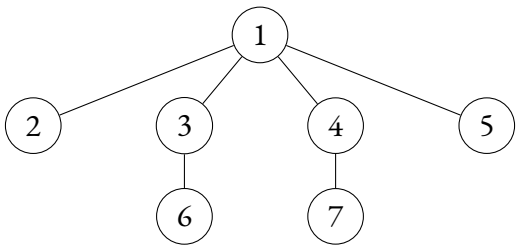
Ebben a fában például:

- `holvan(5)` útja: $5 \to 4 \to 3 \to 2 \to 1$,
- `holvan(6)` útja: $6 \to 3 \to 2 \to 1$,
- `holvan(7)` útja: $7 \to 4 \to 3 \to 2 \to 1$.

Látható, hogy több csúcs mélysége is nagy, így a keresés költséges lehet.

Útösszenyomás után:

Ha például végrehajtjuk a `holvan(5)`, `holvan(6)` és `holvan(7)` műveleteket útösszenyomással, akkor a bejárt csúcsokat közvetlenül a gyökérhez kötjük.



Most:

- `holvan(5)` útja: $5 \to 1$,
- `holvan(6)` útja: $6 \to 3 \to 1$,
- `holvan(7)` útja: $7 \to 4 \to 1$.

A legtöbb csúcs közvetlenül vagy majdnem közvetlenül a gyökérhez kapcsolódik. További `holvan` hívások még tovább lapítják a szerkezetet.

Intuíció: Minden keresés „befektetés” a jövőbe: az aktuális hívás valamivel drágább lehet, de a későbbi keresések egyre gyorsabbá válnak.

Komplexitás útösszenyomással

Csak útösszenyomással: Ha nem használunk rang vagy méret szerinti uniót, csak útösszenyomást, akkor a műveletek amortizált költsége $O(\log n)$.

Méret/rang szerinti unió + útösszenyomás:

A két technika kombinációja adja a klasszikus, optimális Unió–Holvan megoldást.

Ekkor egy művelet amortizált költsége $O(\alpha(n))$, ahol $\alpha(n)$ az inverz Ackermann-függvény.

Az $\alpha(n)$ rendkívül lassan nő: ha $n < 10^{600}$, akkor $\alpha(n) \leq 4$. Ezért a gyakorlatban a műveletek időigénye lényegében konstansnak tekinthető.

3.3.5. Ajánlott implementáció (méret + útösszenyomás)

```
vector<int> szulo, meret;

void init(int n) {
    meret.assign(n + 1, 1);
    szulo.resize(n + 1);
    iota(szulo.begin(), szulo.end(), 0);
}

int holvan(int x) {
    if (szulo[x] == x) return x;
    return szulo[x] = holvan(szulo[x]);
}

void unio(int a, int b) {
    a = holvan(a);
    b = holvan(b);
    if (a == b) return;
    if (meret[a] < meret[b]) swap(a, b);
    szulo[b] = a;
    meret[a] += meret[b];
}
```

4. Alkalmazások

Az Unió-Holvan adatszerkezet legnevezetesebb alkalmazása egy súlyozott gráf minimális feszítőfájának meghatározására szolgáló Kruskal algoritmusban van, viszont a minimális feszítőfákat egy külön tananyagban tárgyaljuk, így itt csak ettől eltérő alkalmazásokat mutatunk.

4.1. Feladat: Road Construction; CSES 1676

Adott `n` város $(1 \leq n \leq 10^5)$, kezdetben nincs köztük út. `m` napon keresztül minden nap egy új utat építenek $(1 \leq m \leq 2 \cdot 10^5)$.

Feladat: minden napra add meg, hogy

- hány összefüggő komponens van (városcsoport, ahol bármelyik két város között van út),
- hány városból áll a legnagyobb komponens.

Bemenet

A `standard bemenet` első sora két egész szám `n` és `m`, azután `m` sor következik, mindegyik két egész szám `a` és `b` $(1 \leq a \neq b \leq n)$, amely azt jelzi, hogy az `a` és `b` városok között új út épül.

Kimenet

A `standard kimenet` minden napra egy sor: két szám, a komponensek száma és a legnagyobb komponens mérete.

Példa

Bemenet	Kimenet
5 3 1 2 1 3 4 5	4 2 3 3 2 3

Korlátok

Időlimit: 1.0 mp.
Memórialimit: 512MiB

4.1.1. Megoldás

Az **Unió-Holvan** ideális adatszerkezet **gráf komponenseinek nyilvántartására**.

Minden egyes útnál tudjuk frissíteni a komponensek számát és a legnagyobb komponens méretét.

Az unio(a,b) műveletben:

- ha két különböző komponenst egyesítettünk, akkor csökkentjük a komponensek számát
- frissítjük a legnagyobb komponens méretét, ha a létrejövő komponens nagyobb nála
- ez konstans extra művelet az unio(a,b)-n belül

Komplexitás: $O(n + m \cdot \alpha(n))$.

C++ implementáció:

```
#include <bits/stdc++.h>
using namespace std;

int n, m, kdb, mx;
vector<int> szulo, meret;

int holvan(int x) {
    if (szulo[x] == x) return x;
    return szulo[x] = holvan(szulo[x]);
}
void unio(int a, int b) {
    a = holvan(a);
    b = holvan(b);
    if (a == b) return;
    if (meret[a] < meret[b]) swap(a, b);
    szulo[b] = a;
    meret[a] += meret[b];
    kdb--;
    mx = max(mx, meret[a]);
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    cin >> n >> m;
    szulo.resize(n+1); meret.assign(n+1, 1);
    iota(szulo.begin(), szulo.end(), 0);
    kdb = n; mx = 1;
    for (int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        unio(a, b);
        cout << kdb << " " << mx << "\n";
    }
}
```

4.2. Feladat: Consecutive Letters; SPOJ CONSEC

Adott egy `S` hosszú string $(1 \leq |S| \leq 200\,000)$, amely csak nagybetűket tartalmaz. Két típusú lekérdezést kapunk:

- `1 i`: Határozd meg a maximális hosszát annak a `[b, e]` intervallumnak, melyre $b \leq i \leq e$ és a `S[b...e]` substring csak `S[i]` betűt tartalmaz.
- `2 i`: Cseréld a `i`-edik karaktert #-re.

Mindkét típusú lekérdezésnél garantált, hogy $S[i] \neq \#$.

Bemenet

A `standard bemenet` első sora maga a string `S`. A következő sor egy egész szám Q $(1 \leq Q \leq 200\,000)$ a lekérdezések számát jelzi. A következő Q sor mindegyikében egy lekérdezés szerepel, formátum: `1 i` vagy `2 i` $(0 \leq i < |S|)$.

Kimenet

A `standard kimenet` minden `1 i` típusú lekérdezésre egy sort tartalmaz: a maximális hosszúságot.

Példa

Bemenet	Kimenet
AABBBCCCC 5 1 0 2 1 1 0 2 2 1 3	2 1 2

Korlátok

Időlimit: 1.0 mp.
Memórialimit: 1536MB

4.2.1. Megoldás

Ötlet: a karakterekből álló, összefüggő szegmenseket halmazokként kezeljük:

- Minden halmazhoz tároljuk a hosszát.
- `holvan(i)` segítségével megtaláljuk, melyik halmazhoz tartozik az `i`-edik karakter.
- `1 i` lekérdezés: visszaadjuk a halmaz hosszát.
- `2 i` lekérdezés (karakter törlése #-re):
  - A törölt karakter miatt a halmaz felhasadhat két részre.
  - A felosztást nem tudjuk hatékonyan csinálni.
  - **Ötlet:** a műveleteket **hátulról visszafelé** is elvégezhetjük, ekkor a szegmensek egyesítése történik → Unió–Holvannal gyorsan kezelhető.

Komplexitás: $O(n + Q \cdot \alpha(n))$

C++ implementáció:

```
string s; cin >> s;
int n = s.size(), q; cin >> q;
vector<pair<int,int>> kerdesek(q);
string akt = s; // másolat a törlésekhez
for (auto& [t, idx] : kerdesek) {
    cin >> t >> idx;
    if (t == 2) akt[idx] = '#';
}
init(n); // unio-holvan inicializálása
for (int i = 1; i < n; i++) {
    if (akt[i] != '#' && akt[i] == akt[i-1]) unio(i-1, i);
}
vector<int> megoldas;
reverse(kerdesek.begin(), kerdesek.end());
for (auto [t, idx] : kerdesek) {
    if (t == 1) {
        megoldas.push_back(meret[holvan(idx)]);
    } else {
        akt[idx] = s[idx];
        if (idx > 0 && akt[idx-1] == akt[idx]) unio(idx-1, idx);
        if (idx + 1 < n && akt[idx+1] == akt[idx]) unio(idx, idx+1);
    }
}
reverse(megoldas.begin(), megoldas.end());
for (int x : megoldas) cout << x << "\n";
```

4.3. Feladat: Legkisebb kizárt; Mester Interaktív feladatok, online algoritmusok 7.

Tartsunk nyilván természetes számok egy halmazát, amelybe folyamatosan érkeznek elemek. Két műveletet kell hatékonyan megvalósítanunk:

- **`Adat(x)`**: Az `x` természetes szám bekerül a halmazba (ha már benne volt, nem történik semmi).
- **`Mex(a)`**: Mi az a legkisebb `b` szám, amelyre $a \leq b$, és `b` **nincs** benne az eddigi halmazban?

Bemenet

A `standard bemenet` egy `n` soros sorozatot tartalmaz $(n \leq 10^5)$, minden sor egy műveletet: `Adat x` vagy `Mex a`, ahol $0 \leq x, a \leq 10^7$.

Kimenet

A `Mex` lekérdezésekre írjuk ki soronként a legkisebb kizárt számot.

Példa

Bemenet	Kimenet

`Adat 0`
`Adat 1`
`Adat 2`
`Adat 4`
`Adat 5`
`Mex 0`
`Mex 4`
`Mex 7`
	`3`
	`6`
	`7`

Korlátok

Időlimit: 0.2 mp.
Memórialimit: 128MiB

4.3.1. Megoldás: Intervallum-ugrás Unió-Holvannal

Ötlet: Az egymást követő, már beérkezett számokat egy halmazba fogjuk, minden halmaz reprezentánsa a szakasz utáni első szabad szám.

- **Inicializálás:** `szulo[i] = i` minden számra.
- `Adat(x)`: Ha `x` bekerül, vonjuk unióba `x`-et a szomszédos `x + 1` elemmel: `unio(x, x+1)`.
- **Irányított unió:** mindig a nagyobb indexű elem legyen az új szülő.
- `Mex(a)`: A válasz a halmaz reprezentánsa: `holvan(a)`.
- **Fontos:** használjunk útösszenyomást a `holvan` függvényben a gyors működéshez.

Komplexitás: $O(n \cdot \alpha(n))$

C++ implementáció:

```cpp
const int M = 10000005;
int szulo[M];
bool inicializalva = false;

void init() {
    if (inicializalva) return;
    inicializalva = true;
    iota(szulo, szulo + M, 0);
}

int holvan(int x) {
    if (szulo[x] == x) return x;
    return szulo[x] = holvan(szulo[x]);
}

void Adat(int x) {
    init();
    szulo[holvan(x)] = holvan(x + 1);
}

int Mex(int a) {
    init();
    return holvan(a);
}
```

4.4. Feladat: Maximum-keresés dinamikus intervallumon

Adott $c_1, \ldots, c_n$ és $l_1, \ldots, l_n$ sorozatok $(0 \leq l_i < i)$. Számítsuk ki minden $i = 1 \ldots n$ esetén az alábbi értéket:

$$a_i = c_i + \max_{l_i \leq j < i} a_j$$

ahol $a_0 = 0$.

Bemenet

A `standard bemenet` első sora az n egész szám $(1 \leq n \leq 10^5)$, következő sor: $c_1 \ldots c_n$, következő sor: $l_1 \ldots l_n$ $(0 \leq l_i < i)$.

Kimenet

A `standard kimenet` egy sort tartalmaz: $a_1 \ldots a_n$ számokat.

Példa

Bemenet	Kimenet
5 2 3 1 4 2 0 0 1 2 1	2 5 6 10 8

4.4.1. Megoldás: Unió-Holvan + Monoton sor

Ötlet: Fenntartunk egy indexlistát, ahol az a_j értékek szigorúan monoton csökkennek.

1. **Törlés (Unió):** Ha egy új a_i kiszámítása után $a_i \geq a_j$, akkor a korábbi j index "feleslegessé" válik. Az Unió-Holvan segítségével minden felesleges indexet a jobbra lévő élő indexhez kötünk.
2. **Lekérdezés (Holvan):** Az l_i indexről indulva a `holvan(`l_i`)` adja meg az első olyan indexet jobbra, amely része a monoton sorozatnak.

4.5. Feladat: Strange Food Chain; SPOJ CHAIN

Három állatfaj van: `A, B, C`, amelyek körbeeszik egymást: `A` $\rightarrow$ `B` $\rightarrow$ `C` $\rightarrow$ `A`. Adott `n` állat $(1 \leq n \leq 50\,000)$ és `k` információ róluk $(1 \leq k \leq 100\,000)$.

Információ típusok:

- **1 x y:** `x` és `y` azonos fajba tartoznak.
- **2 x y:** `x` megeszi `y`-t.

Hamis információk:

1. `x` vagy `y` > `n`
2. `x` azt állítja, hogy megeszi saját magát
3. Ellentmond egy korábbi, igaznak elfogadott állításnak

Feladat: Számoljuk meg, hány információ hamis a `k` darabból!

Bemenet

A `standard bemenet` első sora az `n` és `k`. Következő `k` sor minden sorában `D x y` $(D \in \{1, 2\})$.

Kimenet

A `standard kimenet` egy szám: a hamis állítások száma.

Példa

Bemenet	Kimenet
7 7 1 101 1 2 1 2 2 2 3 2 3 3 1 1 3 2 3 1 1 5 5	3

Korlátok

Időlimit: 1.0 mp.
Memóriakorlát: 1536MiB

4.5.1. Megoldás: Relációs Unió-Holvan

Ötlet: Minden állathoz tároljuk a kapcsolatát a szülőhöz képest a táplálékláncban:

- `0`: azonos faj
- `1`: az állat megeszi a szülőt
- `2`: a szülő eszi az állatot

Műveletek:

- **holvan(x):** Útösszenyomáskor összeadjuk a távolságot mod 3, így kapjuk `x` relációját a gyökérhez képest.
- **unio(d, x, y):** Ha `x` és `y` már ugyanabban a halmazban vannak, ellenőrizzük, hogy a relációik megfelelnek az új információnak. Ha nem, az állítás hamis.
- Ha nincsenek ugyanabban a halmazban, egyesítjük a két fát, és beállítjuk az új gyökérhez viszonyított távolságot.

Komplexitás: $O(k \cdot \alpha(n))$, $\alpha(n)$ az Inverse Ackermann-függvény.

C++ implementáció (részlet)

```cpp
int szulo[50005], tav[50005];

int holvan(int x) {
    if (szulo[x] == x) return x;
    int regi_szulo = szulo[x];
    szulo[x] = holvan(szulo[x]);
    tav[x] = (tav[x] + tav[regi_szulo]) % 3;
    return szulo[x];
}

bool unio(int d, int x, int y) {
    int gy_x = holvan(x), gy_y = holvan(y);
    int elvart_rel = d - 1; // 0: azonos, 1: x eszi y-t
    if (gy_x == gy_y) {
        return (tav[x] - tav[y] + 3) % 3 == elvart_rel;
    }
    szulo[gy_y] = gy_x;
    tav[gy_y] = (tav[x] - tav[y] - elvart_rel + 6) % 3;
    return true;
}
```

5. Unió–Holvan visszagörgetéssel (DSU with rollback)

Motiváció és alapötlet

Bizonyos feladatokban (pl. dinamikus gráfok, offline lekérdezések) nemcsak egyesíteni, hanem **vissza is kell vonni** unió műveleteket.

Ötlet:

- Minden `unio` művelet előtt eltároljuk a módosított változókat.
- Egy **történet verem (stack)** struktúrába mentjük a régi állapotokat.
- A `visszagörget` művelet a mentett értékek alapján visszaállítja az adatokat.

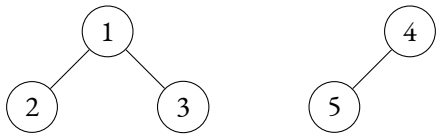
Pillanatkép (snapshot):

- Egy egész szám, amely megadja a történet verem aktuális méretét.
- Később ide visszagörgetve pontosan ezt az állapotot kapjuk vissza.

Fontos: Visszagörgetéses Unió-Holvan esetén **NEM használunk útösszenyomást**, mert az túl sok állapotot változtatna.

Szemléltetés

Kiinduló állapot:



szulo tömb:

x	1	2	3	4	5
szulo[x]	1	1	1	4	4

meret tömb:

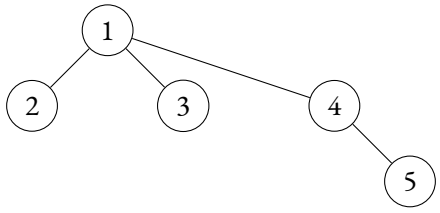
x	1	2	3	4	5
meret[x]	3	1	1	2	1

unio(3,5): a 3-as gyökere 1, az 5-ös gyökere 4, tehát 4-et kötjük 1 alá.

A verembe mentjük:

`(szulo[4] = 4), (meret[1] = 3)`

Unió után:



szulo tömb:

x	1	2	3	4	5
szulo[x]	1	1	1	1	4

meret tömb:

x	1	2	3	4	5
meret[x]	5	1	1	2	1

Történet verem:

szulo[4] = 4
meret[1] = 3

Visszagörgetés: a veremből visszaállítjuk ezeket az értékeket.

5.1. C++ implementáció

```
struct UnioHolvan {
    vector<int> szulo, meret;
    vector<pair<int*, int>> tortenet; // pointer + régi érték

    UnioHolvan(int n) {
        szulo.resize(n+1);
        meret.assign(n+1, 1);
        iota(szulo.begin(), szulo.end(), 0);
    }
    int holvan(int x) {
        while (szulo[x] != x) x = szulo[x];
        return x;
    }
    void unio(int a, int b) {
        a = holvan(a); b = holvan(b);
        if (a == b) return;
        if (meret[a] < meret[b]) swap(a, b);
        // mentjük a változtatásokat
        tortenet.push_back({&szulo[b], szulo[b]});
        tortenet.push_back({&meret[a], meret[a]});
        szulo[b] = a;
        meret[a] += meret[b];
    }
    int pillanatkep() {
        return tortenet.size();
    }
    void visszagorget(int id) {
        while (tortenet.size() > id) {
            *tortenet.back().first = tortenet.back().second;
            tortenet.pop_back();
        }
    }
};
```

5.2. Feladat: Dynamic Connectivity (Dinamikus összefüggőség); CSES 2133

Adott egy irányítatlan gráf `n` csúccsal és `m` éllel. `k` esemény történik:

- **1** `a` `b`: Új él jön létre `a` és `b` között.
- **2** `a` `b`: Egy létező él `a` és `b` között megszűnik.

Feladat: Írd ki az összefüggő komponensek számát a legelső esemény előtt, majd minden egyes esemény után.

Bemenet

A `standard bemenet` első sora három egész szám `n`, `m`, `k` — a csúcsok, élek és események száma. Következő `m` sor: minden sor két egész szám `a` és `b` — az élek. Következő `k` sor: minden sor három egész szám `t`, `a`, `b` (`t = 1` új él, `t = 2` él törlése).

Kimenet

A `standard kimenet` `k + 1` egész szám, először az összefüggő komponensek száma az első esemény előtt, majd minden esemény után.

Példa

Bemenet

```
5 3 3
1 4
2 3
3 5
1 2 5
2 3 5
1 1 2
```

Kimenet

```
2 2 2 1
```

Korlátok

Időlimit: 1.0 mp.
Memórialimit: 512MiB

5.2.1. Megoldási elv

Offline feldolgozás (Divide and Conquer on Queries):

Mivel az éltörlést az Unió-Holvan nem támogatja hatékonyan, az időt használjuk extra dimenzióként.

1. **Élettartam meghatározása:** Minden élhez rendeljünk intervallumokat $[t_{start}, t_{end}]$, amíg a gráfban van.
2. **Szegmensfa az idő felett:** Építsünk egy fát a $[0, k]$ időintervallumra. Minden élt szúrjunk be a fának azon csúcsaiba, amelyek lefedik az élettartamát ($O(\log k)$ csúcs).
3. **Bejárás (DFS):**
   - Lefelé haladva: adjuk hozzá az aktuális csomópontban tárolt éleket a **visszagörgethető Unió-Holvanhoz**.
   - Levélhez érve: az Unió-Holvanban lévő komponensek száma a válasz az adott időpillanatra.
   - Felfelé haladva: **visszagörgetjük** (rollback) az Unió-Holvant a csomópontba lépés előtti állapotra.

Komplexitás: $O((m + k) \cdot \log k \cdot \log n)$.

6. Gyakorló feladatok

6.1. New Roads Queries; CSES 2101

`https://cses.fi/problemset/task/2101/`

Adott `n` város és `m` út, amelyek a megadott sorrendben épülnek meg. Válaszolj meg `q` lekérdezést!

Lekérdezés:

- `a b`: Mi az a legkorábbi időpillanat (nap), amikor az `a` és `b` városok között létrejön az összeköttetés?

Bemenet

A `standard bemenet` első sorában `n m q`, majd `m` sorban az utak `(u, v)`, végül `q` sorban a kérdések.

Kimenet

A `standard kimenet` minden lekérdezésre egy szám: a nap, amikor a két város először ugyanabban a komponensben lesz, vagy `-1`, ha ez sosem történik meg.

Korlátok

$n, m, q \leq 2 \cdot 10^5$

6.2. Two Sets; Codeforces 468B

`https://codeforces.com/contest/468/problem/B`

Adott n különböző pozitív egész szám, valamint két szám: a és b. A feladat a számokat két halmazba (A és B) szétosztani úgy, hogy:

- Ha $x \in A$, akkor $(a - x) \in A$.
- Ha $x \in B$, akkor $(b - x) \in B$.

Kimenet

- Ha nincs ilyen szétosztás: `NO`
- Ha van: `YES` és egy n hosszú 0/1 sorozat, amely jelzi, melyik elem melyik halmazba került.

Korlátok

- $n \leq 10^5$
- $a, b, x_i \leq 10^9$

6.3. BOI 2016 Park

`https://oj.uz/problem/view/BOI16_park`

Egy téglalap alakú parkban ($w \times h$) n darab kör alakú oszlop található. Egy látogató adott r sugarú kör alakú tárggyal szeretne áthaladni a parkon a négy sarokbejárat valamelyike között.

Lekérdezés:

- r sugár és e kezdő sarok esetén melyik sarkokba juthat el a látogató?

Akadályok:

- Két oszlop között akkor nem fér el, ha a távolságuk $< 2r$.
- Egy oszlop és a fal között akkor nem fér el, ha a távolságuk $< 2r$.

Korlátok

- $n \leq 2000, m \leq 10^5$ lekérdezés.
- Az oszlopok nem érnek egymáshoz és nem lógnak ki a parkból.